

Chapter 19: Recovery System

Outline

Failure Classification

Storage Structure

Data Access

Recovery and Atomicity

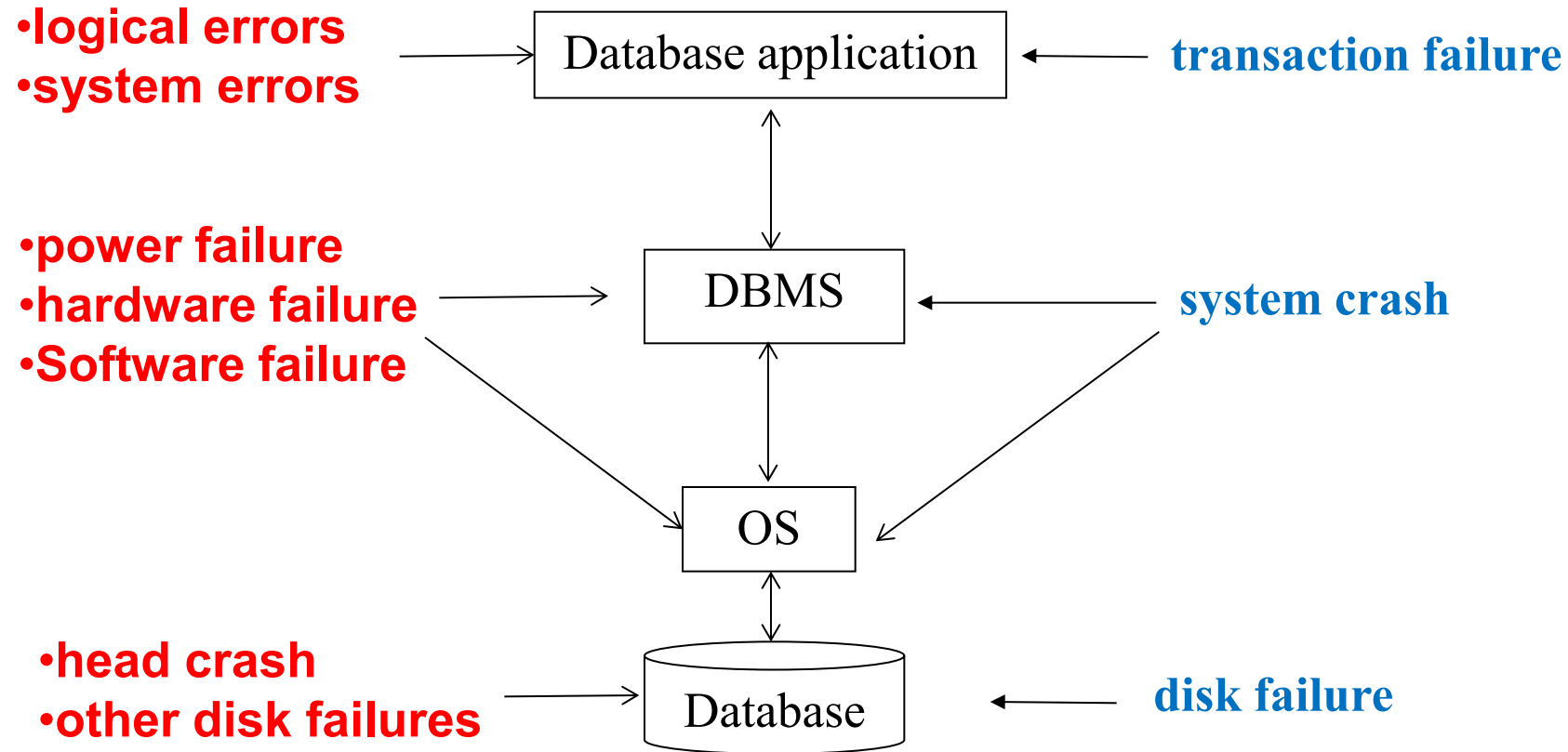
Log-Based Recovery

Remote Backup Systems

Recovery with Early Lock Release and Logical Undo Operations

ARIES Recovery Algorithm

Failure Classification



Failure Classification

Transaction failure :

Logical errors: transaction cannot complete due to some internal error condition

System errors: the database system must terminate an active transaction due to an error condition (e.g., deadlock)

System crash: a power failure or other hardware or software failure causes the system to crash.

Fail-stop assumption: non-volatile storage contents are assumed to not be corrupted by system crash

- ▶ Database systems have numerous integrity checks to prevent corruption of disk data

Disk failure: a head crash or similar disk failure destroys all or part of disk storage

Destruction is assumed to be detectable: disk drives use checksums to detect failures

Storage Structure

Volatile storage:

does not survive system crashes

examples: main memory, cache memory

Nonvolatile storage:

survives system crashes

examples: disk, tape, flash memory,
non-volatile (battery backed up) RAM

but may still fail, losing data

Stable storage:

a **mythical** form of storage that survives all failures

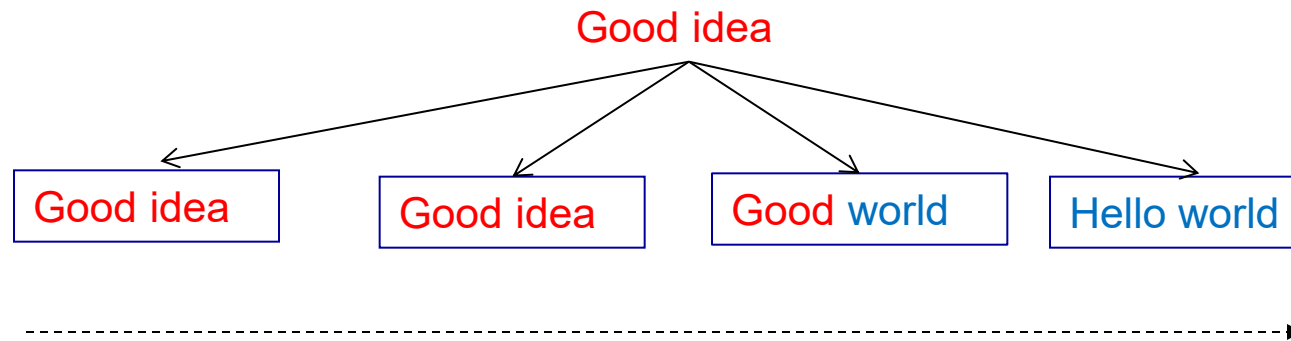
approximated by maintaining multiple copies on distinct nonvolatile media

Stable-Storage Implementation

Maintain multiple copies of each block on separate disks

Failure during data transfer can still result in inconsistent copies

Protecting storage media from failure during data transfer



Database Recovery

Recovery algorithms are techniques to ensure database consistency and transaction atomicity and durability despite failures

Recovery algorithms have two parts

1. Actions taken during normal transaction processing to ensure enough information exists to recover from failures
2. Actions taken after a failure to recover the database contents to a state that ensures atomicity, consistency and durability

We assume that strict two-phase locking ensures no dirty read.

Idempotent(幂等性) : An recovery algorithm is said to be idempotent if executing it several times gives the same result as executing it once.

We study log-based recovery mechanisms in detail

We first present key concepts

And then present the actual recovery algorithm

Log-Based Recovery

Log Records

To ensure atomicity despite failures, we first output information describing the modifications(**log**) to stable storage without modifying the database itself.

A **log** is kept on **stable storage**(稳定存储器).

The log is a sequence of **log records**, and maintains a record of update activities on the database.

When transaction T_i starts, it registers itself by writing a “**start**” log record:

$\langle T_i \text{ start} \rangle$

Before T_i executes **write**(X), writing “**update**” log record

$\langle T_i, X, V_1, V_2 \rangle$

where V_1 is the value of X before the write (the **old value**),

V_2 is the value to be written to X (the **new value**).

When T_i finishes its last statement, writing “**commit**” log record:

$\langle T_i \text{ commit} \rangle$

When T_i complete rollback, writing “**abort**” log record:

$\langle T_i \text{ abort} \rangle$

Log File Example

1. **<T1 start>**
2. **<T1, A, 200, 100>**
3. **<T2 start>**
4. **<T2, B, 300, 200>**
5. **<T3 start>**
6. **<T1, C, 300, 400>**
7. **<T1 commit>**
8. **<T4 start>**
9. **<T3, C, 400, 500>**
10. **<T3 commit>**
11. **<T2, C, 500, 600>**
12. **<T4, A, 100, 700>**
13. **<T2, C, 500>**
14. **<T2, B, 300>**
15. **<T2 abort>**
16. **<T4, A, 100>**
17. **<T4 abort>**

compensation log
(补偿日志)

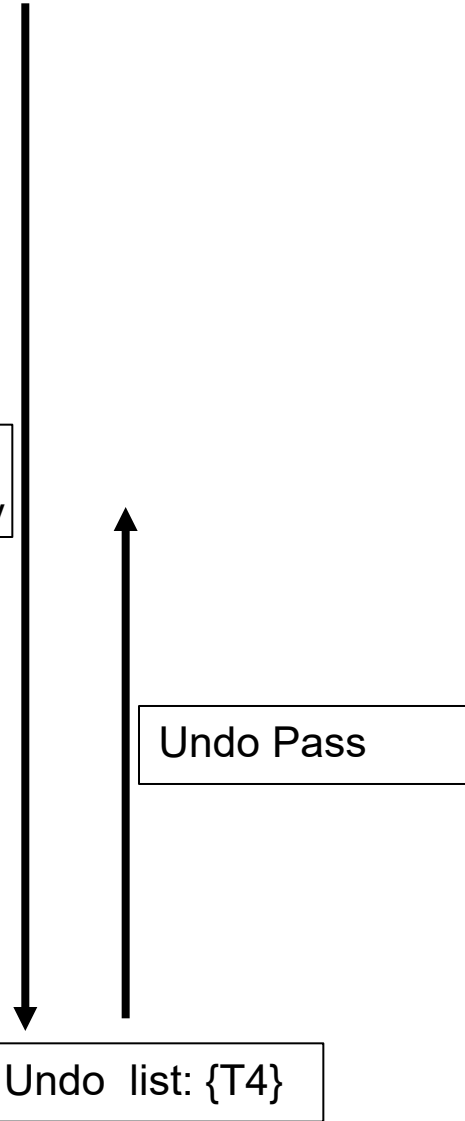
T2 rollback during
normal processing

T4 undone during
recovery

Redo Pass:
Repeating history

Undo Pass

Undo list: {T4}



Write-Ahead Logging (先写日志原则)

Write-Ahead Logging or **WAL**(先写日志原则):

Before a data in main memory is output to the database, the log records pertaining to data must have been output **to stable storage**.

Concurrency Control and Recovery

With concurrent transactions, all transactions share a single disk buffer and a single log file.

A buffer block can have data items updated by one or more transactions

We assume *that if a transaction T_i has modified an item, no other transaction can modify the same item until T_i has committed or aborted*

i.e. the updates of uncommitted transactions should not be visible to other transactions

- ▶ Otherwise how to perform undo if T1 updates A, then T2 updates A and commits, and finally T1 has to abort?

Can be ensured by obtaining exclusive locks on updated items and holding the locks till end of transaction (**strict two-phase locking**)

Recovery with logical undo logging supports early lock release.

Transaction Commit

A transaction is said to have committed when its **commit log record** is output to **stable storage**

all previous log records of the transaction must have been output already

Writes performed by a transaction may still be in the **buffer** when the transaction commits, and may be output later

Undo(撤销) and Redo(重做) Operations

Undo of a log record $\langle T_i, X, V_1, V_2 \rangle$ writes the **old** value V_1 to X

Redo of a log record $\langle T_i, X, V_1, V_2 \rangle$ writes the **new** value V_2 to X

Undo and Redo of Transactions

undo(T_i) restores the value of all data items updated by T_i to their old values, going backwards from the last log record for T_i

- ▶ each time a data item X is restored to its old value V a special log record $\langle T_i, X, V \rangle$ is written out – **compensation log(补偿日志)**
- ▶ when undo of a transaction is complete, a log record **$\langle T_i, \text{abort} \rangle$** is written out.

redo(T_i) sets the value of all data items updated by T_i to the new values, going forward from the first log record for T_i

- ▶ No logging is done in this case

Recovering from Failure

When recovering after failure:

Transaction T_i needs to be **undone** if the log

- ▶ contains the record $\langle T_i \text{ start} \rangle$,
- ▶ but does not contain either the record $\langle T_i \text{ commit} \rangle$ or $\langle T_i \text{ abort} \rangle$.

Transaction T_i needs to be **redone** if the log

- ▶ contains the records $\langle T_i \text{ start} \rangle$
- ▶ and contains the record $\langle T_i \text{ commit} \rangle$ or $\langle T_i \text{ abort} \rangle$

Recovering from Failure - Repeating History

Note that If transaction T_i was undone earlier and the $\langle T_i \text{ abort} \rangle$ record written to the log, and then a failure occurs, on recovery from failure T_i is **redone**

such a redo redoes all the original actions *including the steps that restored old values*

- ▶ Known as **repeating history**(重复历史)
- ▶ Seems wasteful, but simplifies recovery greatly

(Immediate) DB Modification Recovery Example

Below we show the log as it appears at three instances of time.

$\langle T_0 \text{ start} \rangle$
 $\langle T_0, A, 1000, 950 \rangle$
 $\langle T_0, B, 2000, 2050 \rangle$

(a)

$\langle T_0 \text{ start} \rangle$
 $\langle T_0, A, 1000, 950 \rangle$
 $\langle T_0, B, 2000, 2050 \rangle$
 $\langle T_0 \text{ commit} \rangle$
 $\langle T_1 \text{ start} \rangle$
 $\langle T_1, C, 700, 600 \rangle$

(b)

$\langle T_0 \text{ start} \rangle$
 $\langle T_0, A, 1000, 950 \rangle$
 $\langle T_0, B, 2000, 2050 \rangle$
 $\langle T_0 \text{ commit} \rangle$
 $\langle T_1 \text{ start} \rangle$
 $\langle T_1, C, 700, 600 \rangle$
 $\langle T_1 \text{ commit} \rangle$

(c)

Recovery actions in each case above are:

- (a) undo (T_0): B is restored to 2000 and A to 1000, and log records $\langle T_0, B, 2000 \rangle$, $\langle T_0, A, 1000 \rangle$, $\langle T_0, \text{abort} \rangle$ are written out
- (b) redo (T_0) and undo (T_1): A and B are set to 950 and 2050 and C is restored to 700. Log records $\langle T_1, C, 700 \rangle$, $\langle T_1, \text{abort} \rangle$ are written out.
- (c) redo (T_0) and redo (T_1): A and B are set to 950 and 2050 respectively. Then C is set to 600

Checkpoints

Redoing/undoing all transactions recorded in the log can be very slow

1. processing the **entire log** is **time-consuming** if the system has run for a long time
2. we might unnecessarily redo transactions which have already output their updates to the database.

Streamline recovery procedure by periodically performing **checkpointing**

1. **Output all log records** currently residing in main memory onto stable storage.
2. **Output all modified buffer blocks** to the disk.
3. **Write a log record < checkpoint L >** onto stable storage where *L* is a list of all transactions active at the time of checkpoint.

All updates are stopped while doing checkpointing!!!

Checkpoints (Cont.)

During recovery we need to consider only the most recent transaction T_i that started before the checkpoint, and transactions that started after T_i .

1. Scan backwards from end of log to find the most recent **<checkpoint L >** record

Only transactions that are in L or started after the checkpoint need to be redone or undone

Transactions that committed or aborted before the checkpoint already have all their updates output to stable storage.

Some earlier part of the log may be needed for undo operations

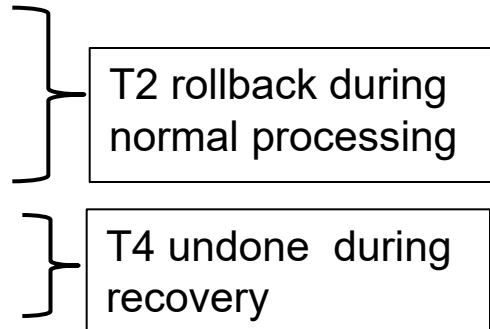
1. Continue scanning backwards till a record **< T_i start>** is found for every transaction T_i in L .

Parts of log prior to earliest **< T_i start>** record above are not needed for recovery, and can be erased whenever desired.

Log File with Checkpoint : Example

1. **<T1 start>**
2. **<T1, A, 200, 100>**
3. **<T2 start>**
4. **<T2, B, 300, 200>**
5. **<T3 start>**
6. **<T1, C, 300, 400>**
7. **<T1 commit>**
8. **<T4 start>**
9. **<T3, C, 400, 500>**
10. **<T3 commit>**
11. **<T2, C, 500, 600>**
12. **<checkpoint {T2,T4}>**
13. **<T4, A, 100, 700>**
14. **<T2, C, 500>**
15. **<T2, B, 300>**
16. **<T2 abort>**
17. **<T4, A, 100>**
18. **<T4 abort>**

compensation log
(补偿日志)



Redo Pass

Undo Pass

Undo list: {T4}

Recovery Algorithm

Logging (during normal operation):

$\langle T_i \text{ start} \rangle$ at transaction start

$\langle T_i, X_j, V_1, V_2 \rangle$ for each update, and

$\langle T_i \text{ commit} \rangle$ at transaction end

Transaction rollback (during normal operation)

Let T_i be the transaction to be rolled back

Scan log backwards from the end, and for each log record of T_i of the form $\langle T_i, X_j, V_1, V_2 \rangle$

- ▶ perform the undo by writing V_1 to X_j
- ▶ write a log record $\langle T_i, X_j, V_1 \rangle$
 - such log records are called **compensation log records(补偿记录)**

Once the record $\langle T_i \text{ start} \rangle$ is found stop the scan and write the log record $\langle T_i \text{ abort} \rangle$

Recovery Algorithm (Cont.)

Recovery from failure: Two phases

Redo phase: replay updates of **all** transactions, whether they committed, aborted, or are incomplete –**repeating history !!!**

Undo phase: undo all incomplete transactions

Redo phase:

1. Find last **<checkpoint L>** record, and set **undo-list** to L .
2. Scan forward from above **<checkpoint L>** record
 1. Whenever a record **$\langle T_i, X_j, V_1, V_2 \rangle$** is found, redo it by writing V_2 to X_j
 2. Whenever a *(compensation) log record* **$\langle T_i, X_j, V_2 \rangle$** is found, redo it by writing V_2 to X_j
 3. Whenever a log record **$\langle T_i, \text{start} \rangle$** is found, add T_i to **undo-list**
 4. Whenever a log record **$\langle T_i, \text{commit} \rangle$** or **$\langle T_i, \text{abort} \rangle$** is found, remove T_i from **undo-list**

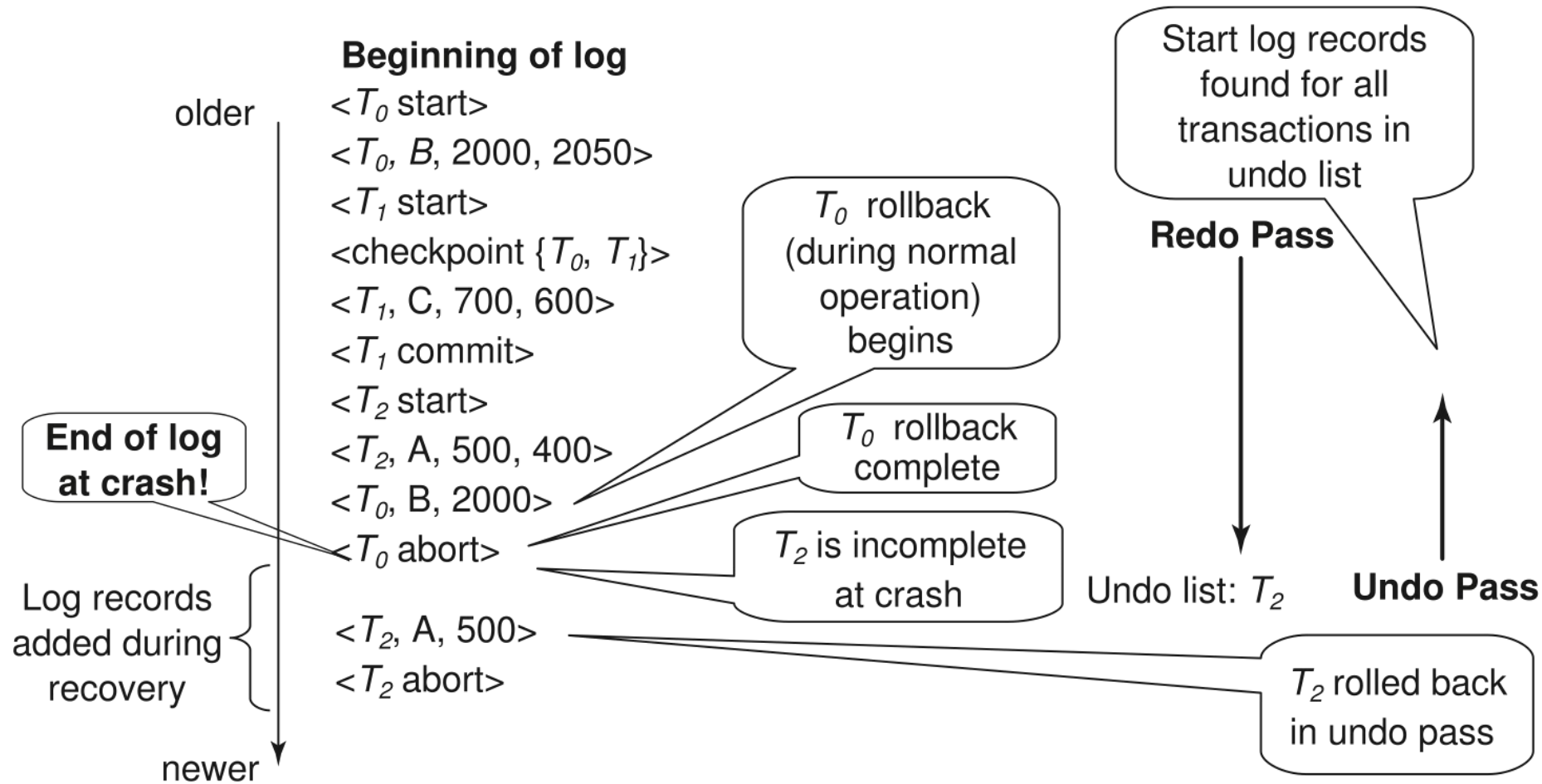
Recovery Algorithm (Cont.)

Undo phase:

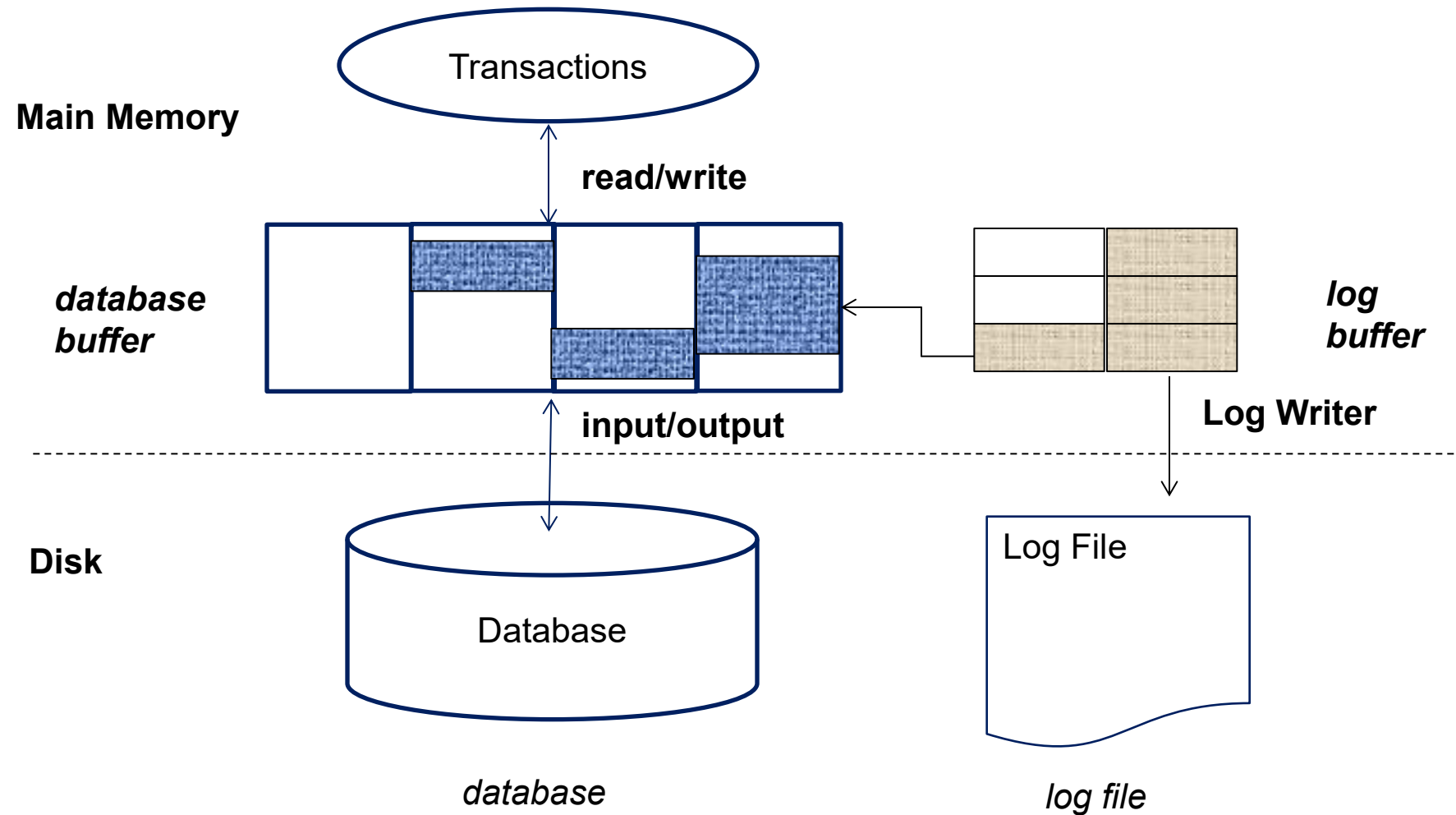
1. Scan log backwards from end
 1. Whenever a log record $\langle T_i, X_j, V_1, V_2 \rangle$ is found where T_i is in undo-list perform same actions as for transaction rollback:
 1. perform undo by writing V_1 to X_j .
 2. write a log record $\langle T_i, X_j, V_1 \rangle$
 2. Whenever a log record $\langle T_i \text{ start} \rangle$ is found where T_i is in undo-list,
 1. Write a log record $\langle T_i \text{ abort} \rangle$
 2. Remove T_i from undo-list
 3. Stop when undo-list is empty
i.e. $\langle T_i \text{ start} \rangle$ has been found for every transaction in undo-list

After undo phase completes, normal transaction processing can commence

Example of Recovery



Log Buffer & Database Buffer



Log Record Buffering

Log record buffering: log records are buffered in main memory, instead of being output directly to stable storage.

Log records are output to stable storage when a block of log records in the buffer is full, or a **log force** operation is executed.

Log force is performed to commit a transaction by forcing all its log records (including the commit record) to stable storage.

Group commit : several log records can be output using a single output operation, reducing the I/O cost.

Log Record Buffering (Cont.)

The rules below must be followed if log records are buffered:

Log records are output to stable storage in the order in which they are created.

Transaction T_i enters the commit state only when the log record $\langle T_i \text{ commit} \rangle$ has been output to stable storage.

Before a block of data in main memory is output to the database, all log records pertaining to data in that block must have been output to stable storage.

- ▶ This rule is called the **write-ahead logging** or **WAL** rule
 - Strictly speaking WAL only requires undo information to be output

Database Buffering

Database maintains an in-memory buffer of data blocks

When a new block is needed, if buffer is full an existing block needs to be removed from buffer

If the block chosen for removal has been updated, it must be output to disk

The recovery algorithm supports the **no-force policy(非强制)**: i.e., updated blocks need not be written to disk when transaction commits

force policy(强制): requires updated blocks to be written at commit

- ▶ More expensive commit

The recovery algorithm supports the **steal policy(窃取策略)**: i.e., blocks containing updates of uncommitted transactions can be written to disk, even before the transaction commits

Fuzzy Checkpointing

To avoid **long interruption** of normal processing during **checkpointing**, allow updates to happen during checkpointing

Fuzzy checkpointing is done as follows:

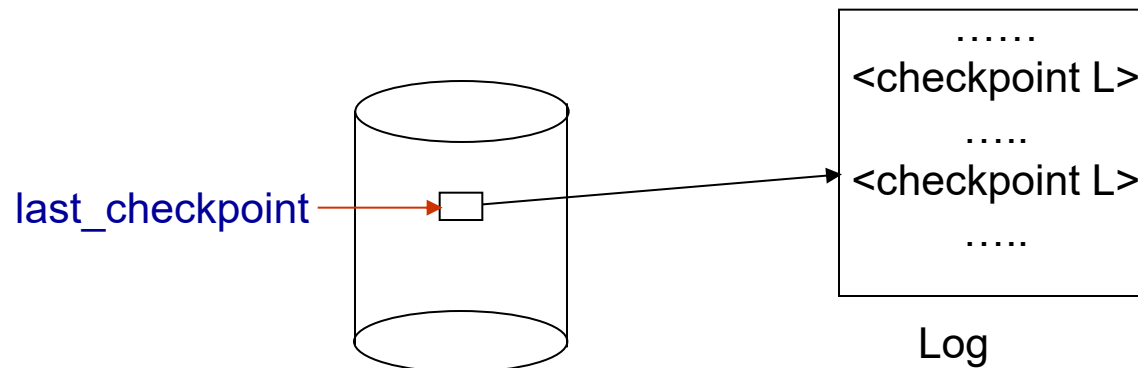
1. Temporarily stop all updates by transactions
2. Write a **<checkpoint L>** log record and force log to stable storage
3. Note **list M of modified buffer blocks**
4. Now permit transactions to proceed with their actions
5. Output to disk all modified buffer blocks in list *M*
 blocks should not be updated while being output
 Follow WAL: all log records pertaining to a block must be output before the block is output
6. Store a pointer to the **checkpoint** record in a fixed position **last_checkpoint** on disk

Fuzzy Checkpointing (Cont.)

When recovering using a fuzzy checkpoint, start scan from the **checkpoint** record pointed to by **last_checkpoint**

Log records before **last_checkpoint** have their updates reflected in database on disk, and need not be redone.

Incomplete checkpoints, where system had crashed while performing checkpoint, are handled safely



Failure with Loss of Nonvolatile Storage

So far we assumed no loss of non-volatile storage

Technique similar to checkpointing used to deal with loss of non-volatile storage

Periodically **dump** the entire content of the database to stable storage

No transaction may be active during the dump procedure; a procedure similar to checkpointing must take place

1. Output all log records currently residing in main memory onto stable storage.
2. Output all buffer blocks onto the disk.
3. Copy the contents of the database to stable storage.
4. Output a record <**dump**> to log on stable storage.

Recovering from Failure of Non-Volatile Storage

To recover from disk failure

1. restore database from most recent dump.
2. Consult the log and redo all transactions that committed after the dump

Can be extended to allow transactions to be active during dump;
known as **fuzzy dump** or **online dump**

Similar to fuzzy checkpointing

Recovery with Early Lock Release and Logical Undo Operations

Recovery with Early Lock Release

Support for high-concurrency locking techniques, such as those used for **B⁺-tree concurrency control**, which release locks early

Supports “logical undo”

Recovery based on “**repeating history**”, whereby recovery executes exactly the same actions as normal processing

Logical Undo Logging

Operations like **B⁺-tree insertions and deletions release locks early**.

They cannot be undone by restoring old values (**physical undo**), since once a lock is released, other transactions may have updated the B⁺-tree.

Instead, insertions (resp. deletions) are undone by executing a deletion (resp. insertion) operation (known as **logical undo**).

For such operations, undo log records should contain the undo operation to be executed

Such logging is called **logical undo logging**, in contrast to **physical undo logging**

- ▶ Operations are called **logical operations**

Other examples:

- ▶ delete of tuple, to undo insert of tuple
 - allows early lock release on space allocation information
- ▶ subtract amount deposited, to undo deposit
 - allows early lock release on bank balance

Physical Redo

Redo information is logged **physically** (that is, new value for each write) even for operations with logical undo

Logical redo is very complicated since database state on disk may not be “operation consistent” when recovery starts

Physical redo logging does not conflict with early lock release

Operation Logging

Operation logging is done as follows:

1. When operation starts, log $\langle T_i, O_j, \text{operation-begin} \rangle$. Here O_j is a unique identifier of the operation instance.
2. While operation is executing, normal log records with physical redo and physical undo information are logged.
3. When operation completes, $\langle T_i, O_j, \text{operation-end}, U \rangle$ is logged, where U contains information needed to perform a logical undo information.

Example: insert of (key, record-id) pair (K5, RID7) into index I9

$\langle T1, O1, \text{operation-begin} \rangle$

....

$\langle T1, X, 10, K5 \rangle$

$\langle T1, Y, 45, \text{RID7} \rangle$

} Physical redo of steps in insert

$\langle T1, O1, \text{operation-end}, (\text{delete I9, K5, RID7}) \rangle$

Operation Logging (Cont.)

If crash/rollback occurs before operation completes:

- the **operation-end** log record is not found, and
- the physical undo information is used to undo operation.

If crash/rollback occurs after the operation completes:

- the **operation-end** log record is found, and in this case
- logical undo is performed using U ; the physical undo information for the operation is ignored.

Redo of operation (after crash) still uses physical redo information.

Transaction Rollback with Logical Undo

Rollback of transaction T_i is done as follows:

Scan the log backwards

1. If a log record $\langle T_i, X, V_1, V_2 \rangle$ is found, perform the undo and log $\langle T_i, X, V_1 \rangle$.
2. If a $\langle T_i, O_j, \text{operation-end}, U \rangle$ record is found
 - ▶ Rollback the operation logically using the undo information U .
 - Updates performed during roll back are logged just like during normal operation execution.
 - At the end of the operation rollback, instead of logging an **operation-end** record, generate a record $\langle T_i, O_j, \text{operation-abort} \rangle$.
 - ▶ Skip all preceding log records for T_i until the record $\langle T_i, O_j, \text{operation-begin} \rangle$ is found

Transaction Rollback with Logical Undo (Cont.)

Transaction rollback, scanning the log backwards (cont.):

3. If a redo-only record is found ignore it
4. If a $\langle T_i, O_j, \text{operation-abort} \rangle$ record is found:
skip all preceding log records for T_i until the record $\langle T_i, O_j, \text{operation-begin} \rangle$ is found.
5. Stop the scan when the record $\langle T_i, \text{start} \rangle$ is found
6. Add a $\langle T_i, \text{abort} \rangle$ record to the log

Some points to note:

Cases 3 and 4 above can occur only if the database crashes while a transaction is being rolled back.

Skipping of log records as in case 4 is important to prevent multiple rollback of the same operation.

Transaction Rollback with Logical Undo

Transaction rollback during normal operation

Beginning of log

$\langle T_0 \text{ start} \rangle$
 $\langle T_0, B, 2000, 2050 \rangle$
 $\langle T_0, O_1, \text{operation-begin} \rangle$
 $\langle T_0, C, 700, 600 \rangle$
 $\langle T_0, O_1, \text{operation-end}, (C, +100) \rangle$
 $\langle T_1 \text{ start} \rangle$
 $\langle T_1, O_2, \text{operation-begin} \rangle$
 $\langle T_1, C, 600, 400 \rangle$
 $\langle T_1, O_2, \text{operation-end}, (C, +200) \rangle$
 $\langle T_0, C, 400, 500 \rangle$
 $\langle T_0, O_1, \text{operation-abort} \rangle$
 $\langle T_0, B, 2000 \rangle$
 $\langle T_0, \text{abort} \rangle$
 $\langle T_1, \text{commit} \rangle$

T_0
decides
to abort

If T_0 aborts before
operation O_1 ends, undo of
update to C will be physical

T_0 has completed operation O_1
on C, releases lower-level
lock; physical undo cannot be
done anymore, logical undo
will add 100 to C

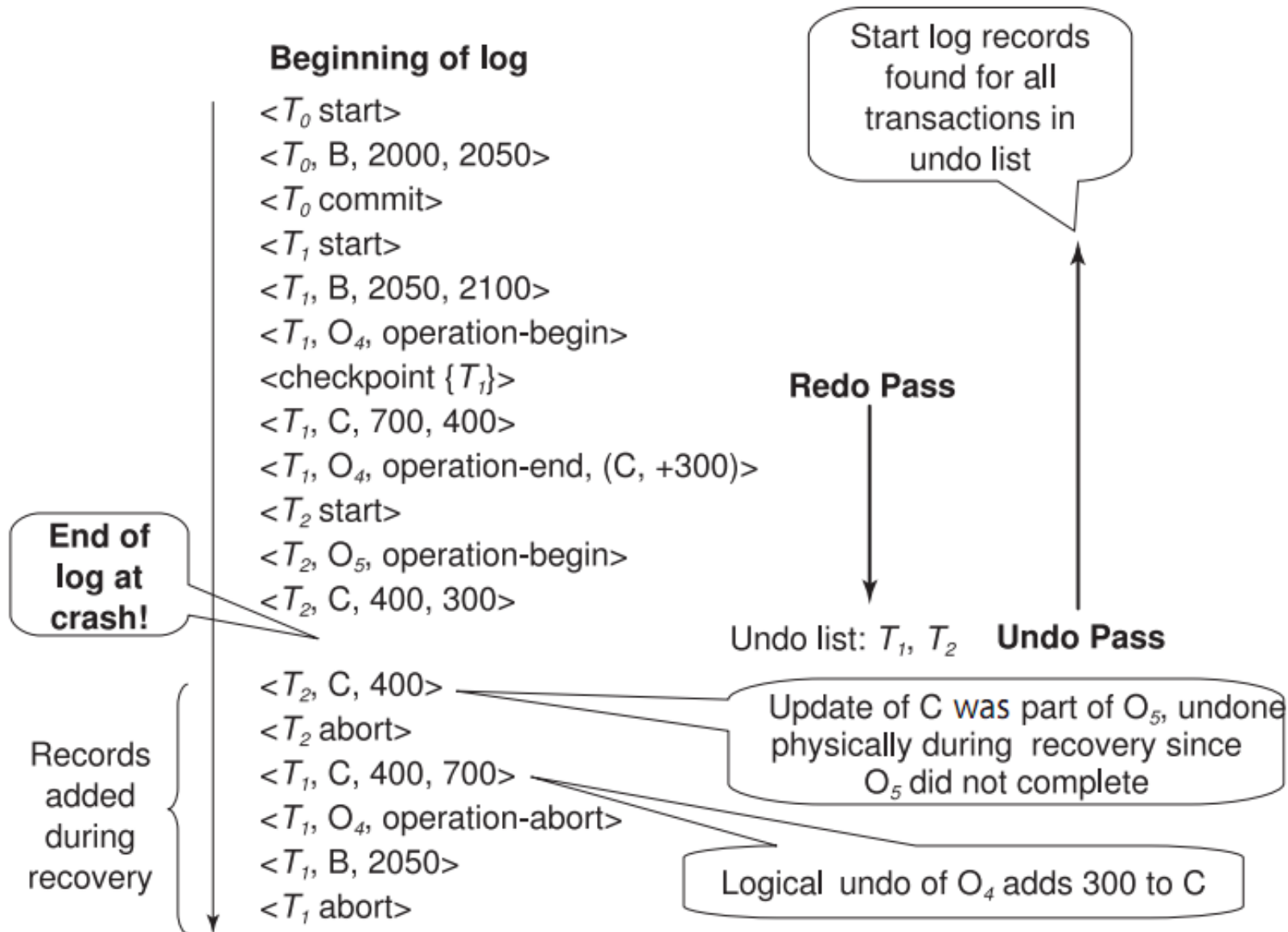
T_1 can update C since T_0 has
released lower-level lock on C

T_1 releases lower-level lock
on C

Logical undo of O_1 adds 100
to C

O_1 undo complete

Failure Recovery with Logical Undo



Recovery Algorithm with Logical Undo

Basically same as earlier algorithm, except for changes described earlier for transaction rollback

1. (**Redo phase**): Scan log forward from last < **checkpoint** L > record till end of log
 1. **Repeat history** by physically redoing all updates of all transactions,
 2. Create an **undo-list** during the scan as follows
 - ▶ **undo-list** is set to L initially
 - ▶ Whenever < T_i **start** > is found T_i is added to **undo-list**
 - ▶ Whenever < T_i **commit** > or < T_i **abort** > is found, T_i is deleted from **undo-list**

This **brings database to state as of crash**, with committed as well as uncommitted transactions having been redone.

Now **undo-list** contains transactions that are **incomplete**, that is, have neither committed nor been fully rolled back.

Recovery with Logical Undo (Cont.)

Recovery from system crash (cont.)

2. (**Undo phase**): Scan log backwards, performing undo on log records of transactions found in *undo-list*.

Log records of transactions being rolled back are processed as described earlier, as they are found

- ▶ Single shared scan for all transactions being undone

When $\langle T_i \text{ start} \rangle$ is found for a transaction T_i in *undo-list*, write a $\langle T_i \text{ abort} \rangle$ log record.

Stop scan when $\langle T_i \text{ start} \rangle$ records have been found for all T_i in *undo-list*

This undoes the effects of incomplete transactions (those with neither **commit** nor **abort** log records). Recovery is now complete.

ARIES Recovery Algorithm

ARIES

ARIES is a state of the art recovery method

Incorporates numerous optimizations to reduce overheads during normal processing and to speed up recovery

The recovery algorithm we studied earlier is modeled after ARIES, but greatly simplified by removing optimizations

Unlike the recovery algorithm described earlier, ARIES

1. Uses **log sequence number (LSN)** to identify log records
 - ▶ Stores LSNs in pages to identify what updates have already been applied to a database page
2. **Physiological redo**
3. **Dirty page table** to avoid unnecessary redos during recovery
4. **Fuzzy checkpointing** that only records information about dirty pages, and does not require dirty pages to be written out at checkpoint time
 - ▶ More coming up on each of the above ...

ARIES Data Structures

ARIES uses several data structures

Log sequence number (LSN) identifies each log record

- ▶ Must be sequentially increasing
- ▶ Typically an offset from beginning of log file to allow fast access
 - Easily extended to handle multiple log files

Page LSN

Log records of several different types

Dirty page table

ARIES Data Structures: Page LSN

Each page contains a **PageLSN** which is the LSN of the last log record whose effects are reflected on the page

To update a page:

- ▶ X-latch the page, and write the log record
- ▶ Update the page
- ▶ Record the LSN of the log record in PageLSN
- ▶ Unlock page

To flush page to disk, must first S-latch page

- ▶ Thus page state on disk is operation consistent
 - Required to support physiological redo

PageLSN is used during recovery to prevent repeated redo

- ▶ Thus ensuring **idempotence**

ARIES Data Structures: Log Record

Each log record contains LSN of previous log record of the same transaction

LSN	TransID	PrevLSN	RedoInfo	UndoInfo
-----	---------	---------	----------	----------

LSN in log record may be implicit

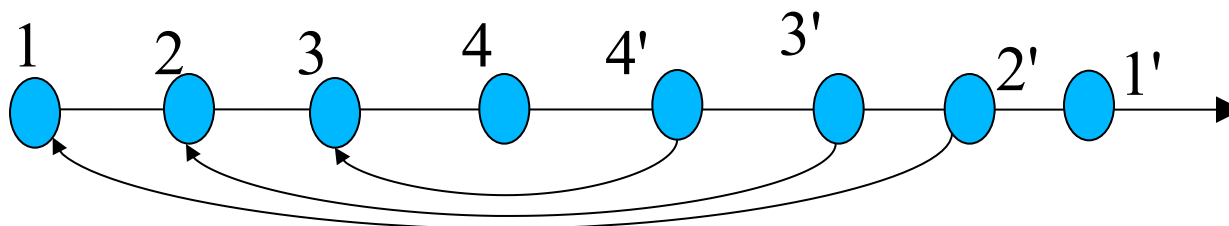
Special redo-only log record called **compensation log record (CLR)** used to log actions taken during recovery that never need to be undone

Serves the role of operation-abort log records used in earlier recovery algorithm

Has a field UndoNextLSN to note next (earlier) record to be undone

- ▶ Records in between would have already been undone
- ▶ Required to avoid repeated undo of already undone actions

LSN	TransID	UndoNextLSN	RedoInfo
-----	---------	-------------	----------



ARIES Data Structures: DirtyPage Table

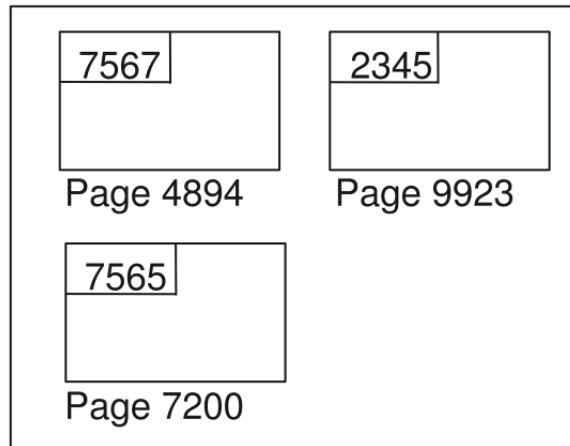
DirtyPageTable

List of pages in the buffer that have been updated

Contains, for each such page

- ▶ **PageLSN** of the page
- ▶ **RecLSN** is an LSN such that log records before this LSN have already been applied to the page version on disk
 - Set to current end of log when a page is inserted into dirty page table (just before being updated)
 - Recorded in checkpoints, helps to minimize redo work

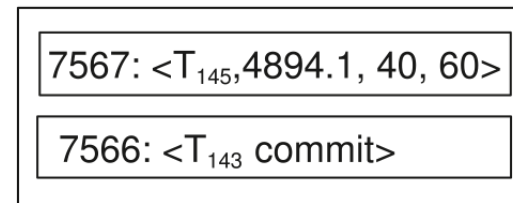
ARIES Data Structures



Database Buffer

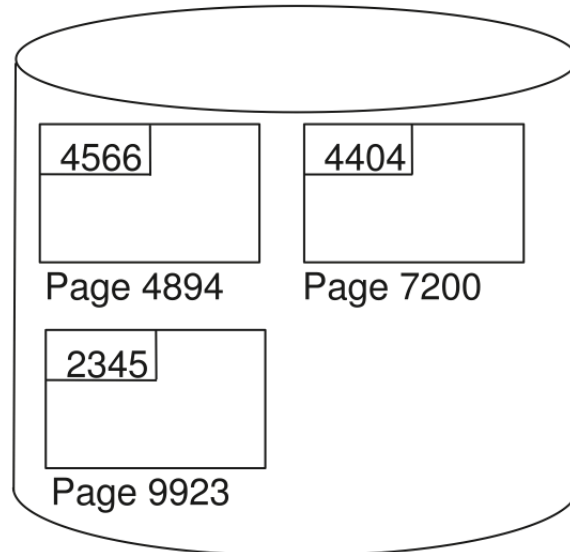
PageID	PageLSN	RecLSN
4894	7567	7564
7200	7565	7565

Dirty Page Table

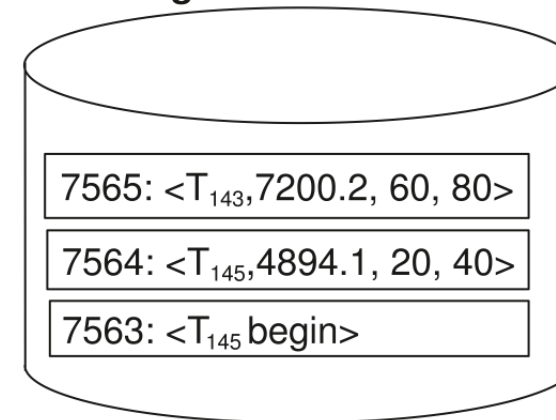


Log Buffer (PrevLSN and UndoNextLSN fields not shown)

Stable data



Stable log



ARIES Data Structures: Checkpoint Log

Checkpoint log record

Contains:

- ▶ DirtyPageTable and list of active transactions
- ▶ For each active transaction, LastLSN, the LSN of the last log record written by the transaction

Fixed position on disk notes LSN of last completed checkpoint log record

Dirty pages are not written out at checkpoint time

- ▶ Instead, they are flushed out continuously, in the background

Checkpoint is thus very low overhead

can be done frequently

ARIES Recovery Algorithm

ARIES recovery involves three passes

Analysis pass: Determines

Which transactions to undo (**undo-list**)

Which pages were dirty (disk version not up to date) at time of crash

RedoLSN: LSN from which redo should start

Redo pass:

Repeats history, redoing all actions from **RedoLSN**

- ▶ RecLSN and PageLSNs are used to avoid redoing actions already reflected on page

Undo pass:

Rolls back all incomplete transactions

- ▶ Transactions whose abort was complete earlier are not undone
 - Key idea: no need to undo these transactions: earlier undo actions were logged, and are redone as required

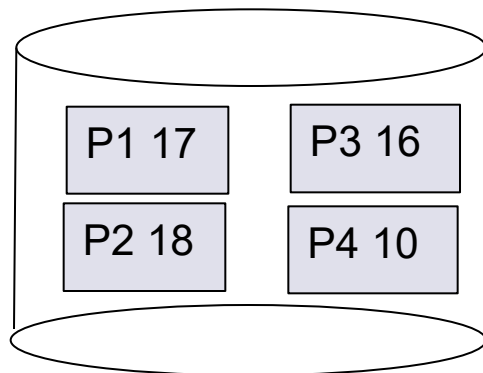
ARIES Data Structures

Dirty Page Table

PageID	PageLSN	RecLSN
P1	22	20
P2	21	21
P3	19	19

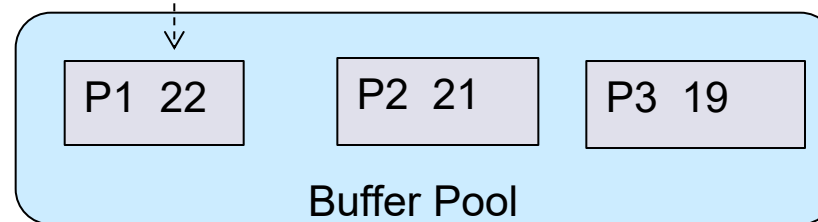
Active TXN Table

Tno	LastLSN
T1	22
T2	20
T3	21



Page LSNs on disk

PageLSN



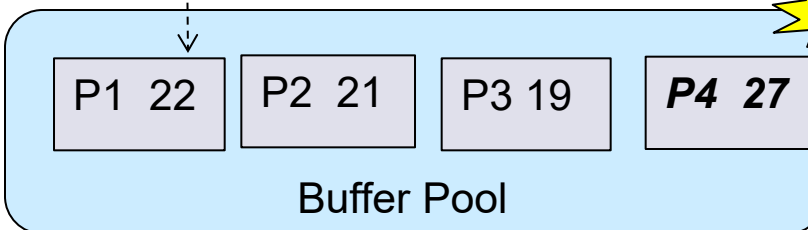
LSN	PrevLSN	TNO	TYPE	PID	len	offset	before	after
15		T1	U	P1	3	21	ABC	DEF
16		T2	U	P3	3	41	HIJ	KLM
17	16	T2	U	P1	3	20	GDE	QRS
18	15	T1	U	P2	3	21	TUV	WXY
19		T3	U	P3	3	51	COM	EDU
20	17	T2	U	P1	3	30	EDU	ORG
21	19	T3	U	P2	3	60	SCI	ART
22	18	T1	U	P1	3	30	ORG	COM
23	BEGIN_C							
24	END_CKP{P1:20,P2:21,P3:19} {T1:22,T2:20,T3:21}							

ARIES Data Structures(cont.)

Dirty Page Table

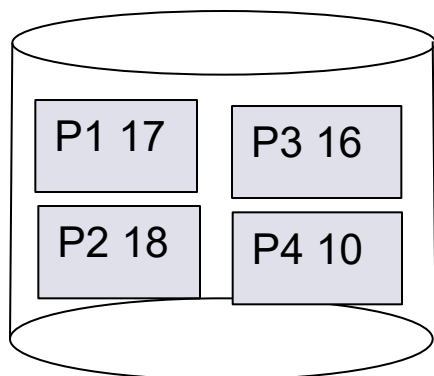
Page	PageLSN	RecLSN
P1	22	20
P2	21	21
P3	19	19
P4	27	27

PageLSN



Active TXN Table

Tno	LastLSN
T2	27
T3	21



Page LSNs on disk

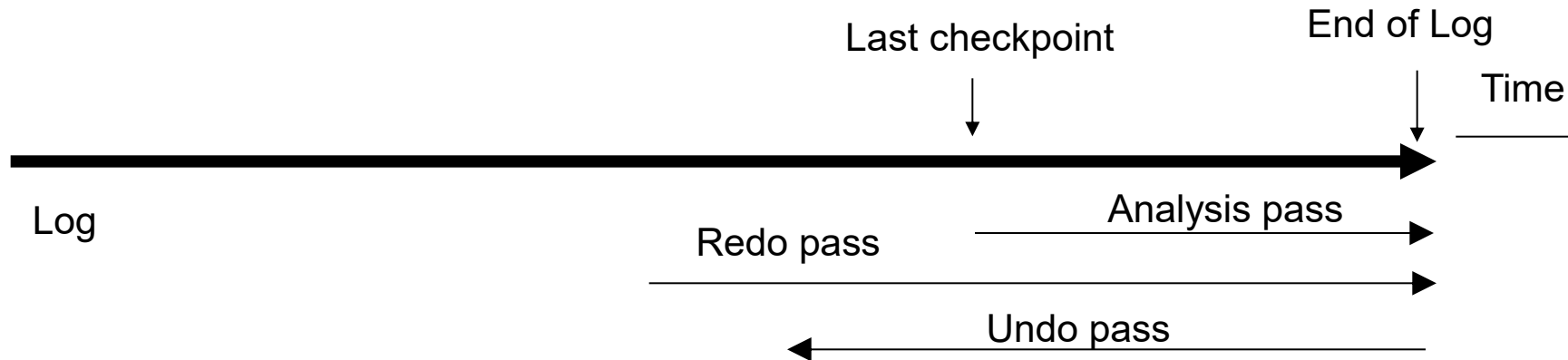
LSN	PrevLSN	TNO	TYPE	PID	len	offset	before	after
15	T	T1	U	P1	3	21	ABC	DEF
16	T	T2	U	P3	3	41	HIJ	KLM
17	16	T2	U	P1	3	20	GDE	QRS
18	15	T1	U	P2	3	21	TUV	WXY
19	T	T3	U	P3	3	51	COM	EDU
20	17	T2	U	P1	3	30	EDU	ORG
21	19	T3	U	P2	3	60	SCI	ART
22	18	T1	U	P1	3	30	ORG	COM
23		BEGIN_C						
24		END_CKP{P1:20,P2:21,P3:19} {T1:22,T2:20,T3:21}						
25	22	T1	C					
26	25	T1	E					
27	20	T2	U	P4	3	40	XYX	ABC

Aries Recovery: 3 Passes

Analysis, redo and undo passes

Analysis determines where redo should start

Undo has to go back till start of earliest incomplete transaction



ARIES Recovery: Analysis

Analysis pass

Starts from last complete checkpoint log record

Reads **DirtyPageTable** from log record

Sets **RedoLSN** = min of RecLSNs of all pages in DirtyPageTable

- ▶ In case no pages are dirty, RedoLSN = checkpoint record's LSN

Sets **undo-list** = list of transactions in checkpoint log record

Reads LSN of last log record for each transaction in undo-list from checkpoint log record

Scans forward from checkpoint

.. Cont. on next page ...

ARIES Recovery: Analysis (Cont.)

Analysis pass (cont.)

Scans forward from checkpoint

If any log record found for transaction not in undo-list, adds transaction to **undo-list**

Whenever an update log record is found

- ▶ If page is not in **DirtyPageTable**, it is added with RecLSN set to LSN of the update log record

If transaction end log record found, delete transaction from **undo-list**

Keeps track of last log record for each transaction in undo-list

- ▶ May be needed for later undo

At end of analysis pass:

RedoLSN determines where to start redo pass

RecLSN for each page in DirtyPageTable used to minimize redo work

All transactions in **undo-list** need to be rolled back

ARIES Redo Pass

Redo Pass:

Repeats history by replaying every action not already reflected in the page on disk, as follows:

Scans forward from RedoLSN. Whenever an update log record is found:

1. If the page is not in DirtyPageTable or the LSN of the log record is less than the RecLSN of the page in DirtyPageTable, then skip the log record
2. Otherwise fetch the page from disk. If the PageLSN of the page fetched from disk is less than the LSN of the log record, redo the log record

NOTE: if either test is negative the effects of the log record have already appeared on the page. First test avoids even fetching the page from disk!

ARIES Undo Actions

When an undo is performed for an update log record

Generate a CLR containing the undo action performed (actions performed during undo are logged physically or physiologically).

- ▶ CLR for record n noted as n' in figure below

Set UndoNextLSN of the CLR to the PrevLSN value of the update log record

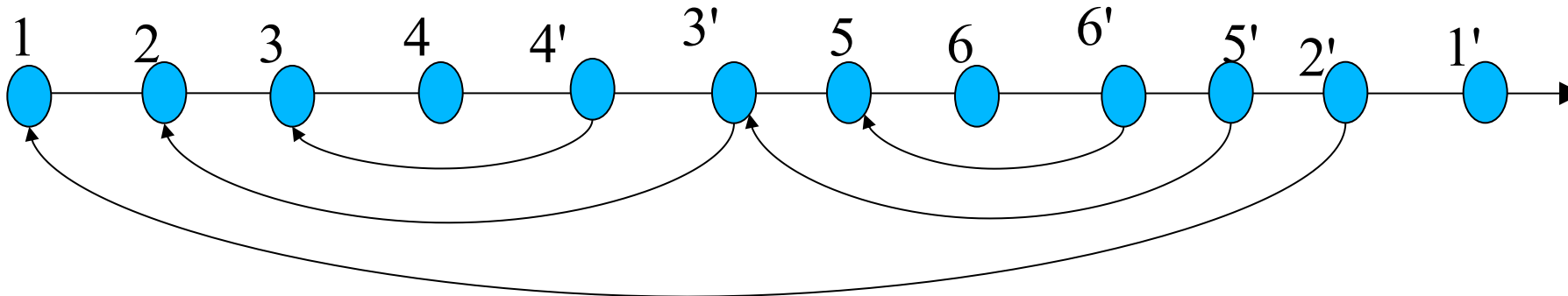
- ▶ Arrows indicate UndoNextLSN value

ARIES supports **partial rollback**

Used e.g. to handle deadlocks by rolling back just enough to release reqd. locks

Figure indicates forward actions after partial rollbacks

- ▶ records 3 and 4 initially, later 5 and 6, then full rollback



ARIES: Undo Pass

Undo pass:

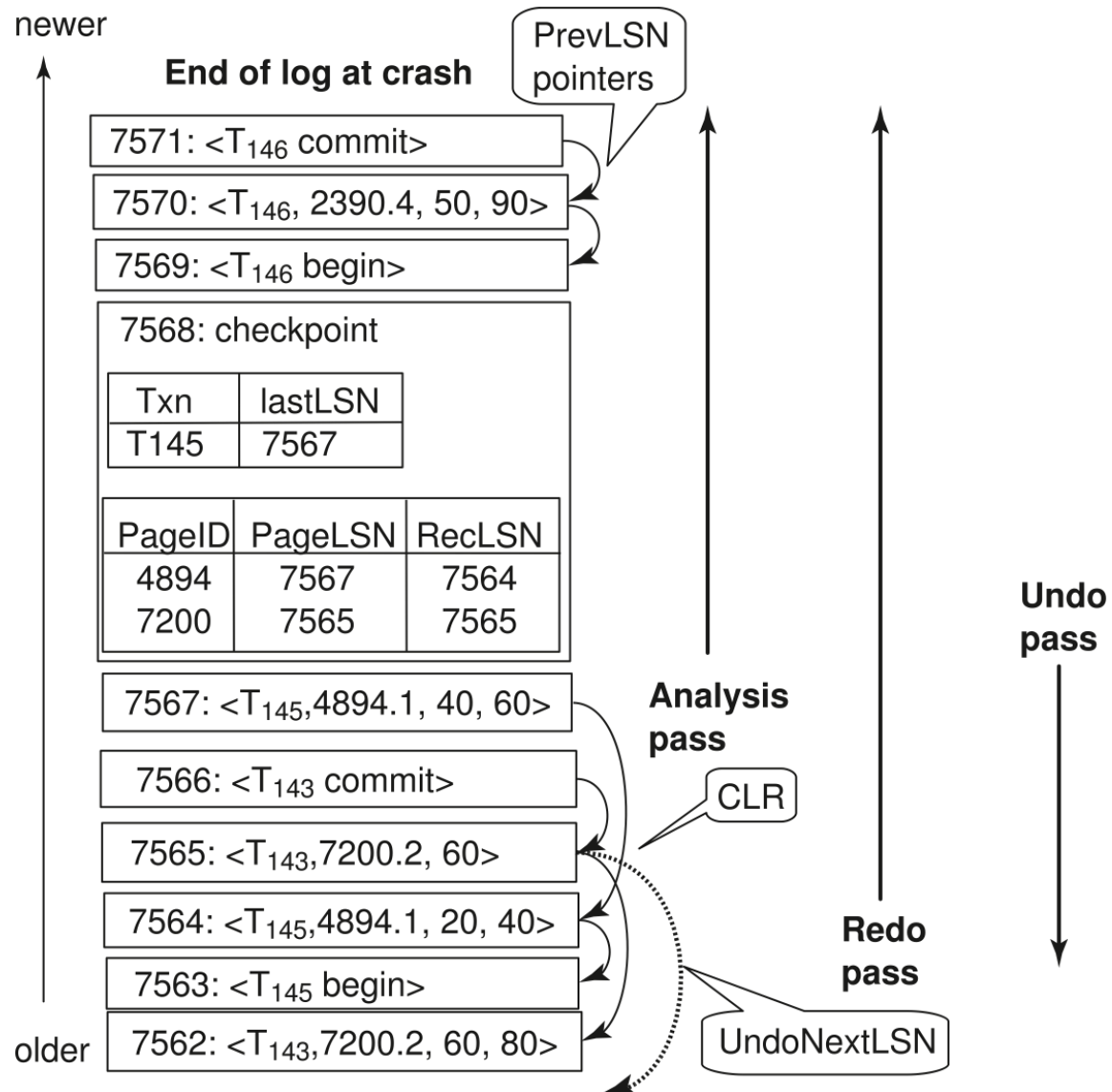
Performs backward scan on log undoing all transaction in [undo-list](#)

Backward scan optimized by skipping unneeded log records as follows:

- ▶ Next LSN to be undone for each transaction set to LSN of last log record for transaction found by analysis pass.
- ▶ At each step pick largest of these LSNs to undo, skip back to it and undo it
- ▶ After undoing a log record
 - For ordinary log records, set next LSN to be undone for transaction to PrevLSN noted in the log record
 - For compensation log records (CLRs) set next LSN to be undo to [UndoNextLSN](#) noted in the log record
 - » All intervening records are skipped since they would have been undone already

Undos performed as described earlier

Recovery Actions in ARIES



End of Chapter 19